Comp 533 - Assignment 5: Basic and Meta Object Replication Using GIPC RPC Abstractions

Date Assigned: March 21, 2025

Early Completion Date: Tuesday, April 8, 2025, 11:55 pm (+5% extra credit)

Completion Date: Friday, April 11, 2025, 11:55 pm

Late Date: Friday, April 18, 2025, 11:55 pm (-10% late penalty)

Scary Late Date: Friday, April 25, 2025, 11:55pm (-30% late penalty)

The goals of the assignment are (a) to support multiple forms of RPC-based inter process communication mechanisms and switch between them dynamically, and (b) to replicate meta replication state: state maintained by the system to support replication of non-meta, basic state.

You will *add* to your solution, another RPC mechanism, GIPC. The choice of the RPC abstraction to be used - RMI or GIPC - will be decided by a meta object called IPC mechanism (choice). The idea of changing aspects of the distributed system dynamically, depending on the application and other features, is of particular importance in big data crunching as overheads (such as RMI blocking procedure calls) should be paid only if necessary.

In the previous assignment, the simulation and command processor objects are replicated (Recall that an object is replicated if multiple processes have local copies of it that are kept consistent with each other.) You will replicate another object here – the IPC mechanism (choice) - which is a meta replication object in that its purpose is to aid in the process of replicating other, basic objects, the simulation and command processor.

The client will be able to replicate basic objects (simulation and command processor) using both RMI and GIPC. The IPC mechanism choice meta object will be replicated using only RMI in this and future assignments.

GIPC RPC

In this part, you must allow the communication among clients and the server to be done using not only RMI but also GIPC.

RMI vs. GIPC

GIPC (for Generalized Inter-Process Communication) is different from RMI in many ways. The differences relevant to this assignment are that (a) the server and registry are integrated in one process, (b) proxies are always created in the client process, (c) each client must have a globally unique string name (which can be hostname concatenated with some host-relative name), and (d) calls to procedures – methods that do not return values – are asynchronous and do not block. As in RMI, calls to functions are synchronous and block until the value is returned. As in RMI, proxies are automatically created when Remote parameters are passed in calls to remote methods. In RMI, the passed parameters need to be exported by exportObject. exportObject is not necessary in GIPC.

GIPC proxies are different from RMI proxies, and use different data communication channels (NIO in GIPC and sockets in RMI) for communicating messages. Thus, a server needs to maintain separate data structures for storing them, and choose between them based on the state of the IPC mechanism meta object.

GIPC has an API that is analogous to the one offered by RMI. The following table shows the similarities and differences (in bold).

RMI	GIPC
java.rmi.registry.LocateRegistry	inputport.rpc.GIPCLocateRegistry
java.rmi.registry.Registry	inputport.rpc.GIPCRegistry
java.rmi.server.UnicastRemoteObject	n/a
Registry LocateRegistry.createRegistry(int)	GIPCRegistry GIPCLocateRegistry.createRegistry(int)
Registry.rebind(String, Object)	GIPCRegistry.rebind(String, Object)
UnicastRemoteObject.exportObject(String, int)	n/a
LocateRegistry.getRegistry(String, int)	GIPCLocateRegistry.getRegistry(String, int, String)
Object Registry.lookup(String)	Object Registry.lookup(**Class**, String) // To get class, use syntax <interfaceName>.class
n/a	**GIPCRegistry.getInputPort().addConnectionListener(ConnectionListener)**
checked Java.rmi.RemoteException	**unchecked** inputport.rpc.duplex.GIPCRemoteException

The GIPC getRegistry() method takes not only the name of the host and port of the server (the first two arguments) but also the unique client name as a third argument. As proxies are created in the client, the lookup method takes an extra argument specifying the interface (represented as a Class object) to be used to create the proxy. The last row is a

facility to add a listener for connection and disconnection events. Ignore the warning message you get about using interfaces to create proxies, if you have warnings enabled.

In GIPC, look at the package, examples.gipc.counter.simple, for an example of a server whose methods are invoked by two different clients. The server is ASimpleGIPCRegistryAndCounterServer, and the clients are ASimpleGIPCCounterClient0 and ASimpleGIPCCounterClient1. The code in this package mirrors and, in fact uses, the RMI example, mentioned in the previous assignment, in package examples.rmi.counter.simple. A connection listener (in another package) listens for connection and disconnection events. An *unchecked* GIPCRemoteException is thrown when a (synchronous) call to a remote function fails.

Specifying GIPC Ports in the Server and Client

GIPC requires the server to be explicitly bound to a port and the clients to know about this port. The port number should not be hardwired - it should be a main arg.

The method, getGIPCServerPort(), in ServerArgsProcessor specifies the GIPC port number to be used by the server in its GIPC createRegistry() call. Similarly, the method, getGIPCPort(), in ClientArgsProcessor determines the GIPC port number to be given in the client getRegistry() call.

As in A4, you do not have to specify the main args in this assignment because of the provided defaults.

IPC Switch Dynamic Parameter

The predefined type, IPCMechanism, is used to determine the IPC mechanism.

```
package util.interactiveMethodInvocation;
public enum IPCMechanism {
	NIO,
	RMI,
	GIPC
}
```

The IPC mechanism will be changed interactively using a console command of the form:

```
i <IPC Mechanism>
```

Thus, to change the current IPC mechanism from RMI to GIPC a user should be able to execute the command :

```
i GIPC
```

To change it back to RMI, the user executes the command:

```
i RMI
```

It is your task, of course, to intercept and react to these commands. If your client and server class is an extension of the previous assignment client and server class

respectively,,then it makes the following calls in a client and server parameter listener object:

```
SimulationParametersControllerFactory.getSingleton().addSimulationParame
terListener(this);
        // use the calling back library
SimulationParametersControllerFactory.getSingleton().processCommands();
```

The class of the listener objects (client or server class) extends AnAbstractSimulationParametersBean.

The method

void ipcMechanism(IPCMechanism newValue)

inherited from AnAbstractSimulationParametersBean is called when an IPC changing command is executed by the user. It is your task to override the method in the client and server to:

- change an internal local copy of a replicable object/variable whose logical, traced name, is “ipc_mechanism”.
- choose the IPC mechanism used to communicate simulation commands based on this object/variable.
- use the ProposalMade trace class to trace the execution of the interactive command.
- use the ProposalStateSet trace class to trace the setting of the internal object/variable

The tracing is shown below:

```
i GIPC
public default void
assignments.util.inputParameters.SimulationParametersListener.ipcMechani
sm(util.interactiveMethodInvocation.IPCMechanism)[GIPC]

I***@928657739901300 {main}(ProposalMade)
EvtSrc(Your_Client_Simulation_Class_Name)  ipc_mechanism,-1.0=GIPC

I***@928660841007900 {main}(ProposedStateSet)
EvtSrc(Your_Client_Simulation_Class_Name)  ipc_mechanism,-1.0=GIPC

Enter (prefixes of) print, quit or a method and its parameters separated
by commas:
```

The server and clients will be ready to receive messages using any of the supported IPC mechanisms. The current IPC mechanism will be used only by the sending client objects (out couplers), which are observers of the command processor and also senders of the data to the server.

Broadcast Meta State Dynamic Parameter

Based on a dynamic user parameter, you should be able to process a change to the local copy of the IPC mechanism in one client by broadcasting, using RMI, the new IPC mechanism to all other processes (server and other clients), all of which then change the mode or IPC Mechanism. This means the server must also have a variable/object to hold the value of the current IPC mechanism.

Override the following method in the client program:

void broadcastMetaState(boolean newValue)

inherited from AnAbstractSimulationParametersBean to allow the user to change this parameter from a client console:

```
b true
public default void
assignments.util.inputParameters.SimulationParametersListener.broadcastM
etaState(boolean)[true]
```

When this parameter is set to true by the user, each change to the IPC Mechanism is broadcast. Changing the parameter to true does not result in any message to be sent, it only affects future changes to the IPC mechanism. The default value of this parameter is false:

Trace at changing client:

```
Enter (prefixes of) print, quit or a method and its parameters separated
by commas:
i GIPC
public default void
assignments.util.inputParameters.SimulationParametersListener.ipcMechani
sm(util.interactiveMethodInvocation.IPCMechanism)[GIPC]

I***@929170238357500 {main}(ProposalMade)
EvtSrc(Your_Client_Simulation_Class_Name)  ipc_mechanism,-1.0=GIPC

I***@929176310750100 {RMI TCP Connection(12)-
192.168.86.218}(ProposedStateSet)
EvtSrc(Your_Client_Simulation_Class_Name)  ipc_mechanism,-1.0=GIPC

I***@929170238964000 {main}(RemoteProposeRequestSent)
EvtSrc(Your_Client_Simulation_Class_Name)  ipc_mechanism,-1.0=GIPC

I***@929170305267200 {RMI TCP Connection(12)-
Enter (prefixes of) print, quit or a method and its parameters separated
by commas:
```

Trace at the server:

```
I***@930591623638700 {RMI TCP Connection(4)-
192.168.86.218}(RemoteProposeRequestReceived)
EvtSrc(Your_RMI_Relayer_Class_Name)  ipc_mechanism,-1.0=GIPC
```

Trace at a receiving client:

```
I***@930591636402500 {RMI TCP Connection(2)-
192.168.86.218}(ProposalLearnedNotificationReceived)
EvtSrc(Your_Coupler_Class_Name)  ipc_mechanism,-1.0=GIPC
I***@930591636842300 {RMI TCP Connection(2)-
192.168.86.218}(ProposedStateSet)
EvtSrc(Your_Client_Simulation_Class_Name)  ipc_mechanism,-1.0=GIPC
```

Make sure that a local rather than remote call is made in the client/server to check the current value of the broadcast mode or ipc mechanism. As mentioned earlier, this meta state, like the Halloween simulation state, is replicated – each process has a copy of it that is kept consistent with other copies.

The following trace shows what happens when a command is entered at a client whose current IPC mechanism is GIPC:

Trace at client invoking command:

```
s "move 100 -100"
public default void
assignments.util.inputParameters.SimulationParametersListener.simulation
Command(java.lang.String)[move 100 -100]

I***@1033152029090400 {main}(SetProperty)
EvtSrc(Your_Halloween_Processor_Class_Name)  InputString = move 100 -100

I***@1033152035215700 {main}(CommandSubmitted)
EvtSrc(Your_Halloween_Processor_Class_Name)  Command:move 100 -100

I***@1033152468831500 {main}(NotifiedPropertyChangeEvent)
EvtSrc(Your_Halloween_Processor_Class_Name)
java.beans.PropertyChangeEvent[propertyName=InputString; oldValue=;
newValue=move 100 -100; propagationId=null;
source=Your_Halloween_Processor_Class_Name]->[,
nio.manager.simulation.client.AnNIOManagerSimulationClientOutCoupler@7db
47323, rmi.simulation.client.AnRMISimulationOutCoupler@3c4059a,
rmi.simulation.client.AnRMISimulationOutCoupler@702cfbde]

I***@1033152519171500 {main}(RemoteProposeRequestSent)
EvtSrc(Your_Coupler_Class_Name)  command,-1.0=move 100 -100
I***@1033152549832100 {main}(SocketChannelWriteRequested)
EvtSrc(AnNIOManager)  java.nio.HeapByteBuffer[pos=0 lim=572 cap=848],
[inputport.datacomm.duplex.buffer.nio.AnNIODuplexBufferClientInputPortDr
iver@738a39cc] java.nio.channels.SocketChannel[connected
local=/127.0.0.1:61525 remote=localhost/127.0.0.1:10000]
```

Notice that we see certain traces we do not get when RMI is used such as SocketChannelWriteRequested. These are created by the underlying NIO mechanism used in GIPC to send RPC call data.

Trace at server:

```
I***@1033152670087800 {Asynchronous Received Call
Invoker}(ReceivedCallInitiated) EvtSrc(ADuplexReceivedCallInvoker)
Generic Client:0.23907034445495323--
```

```
>(Your_RMI_Relayer_Class_Name).public abstract void
rmi.simulation.common.RMISimulationRelayer.broadcast(java.lang.String,ja
va.lang.String) throws java.rmi.RemoteException([Generic
Client:0.23907034445495323, move 100 -100])

I***@1033152682770500 {Asynchronous Received Call
Invoker}(RemoteProposeRequestReceived)
EvtSrc(Your_RMI_Relayer_Class_Name)  Generic Client:0.23907034445495323-
-> command,-1.0=move 100 -100


I***@1033152688533700 {Asynchronous Received Call
Invoker}(ProposalLearnedNotificationSent)
EvtSrc(Your_RMI_Relayer_Class_Name)  command,-1.0=move 100 -100
```

Again, we see some GIPC-specific traces such as ReceivedCallInitiated.

Trace at a receiving client:

```
I***@1033153266696800 {Asynchronous Received Call
Invoker}(ReceivedCallInitiated) EvtSrc(ADuplexReceivedCallInvoker)
Default Server-->(Your_Coupler_Class_Name).public abstract void
rmi.simulation.common.RMISimulationInCoupler.processCommand(java.lang.St
ring) throws java.rmi.RemoteException([move 100 -100])

I***@1033153289540400 {Asynchronous Received Call
Invoker}(ProposalLearnedNotificationReceived)
EvtSrc(Your_Coupler_Class_Name)  Default Server--> broadcast_mode,-
1.0=move 100 -100

I***@1033153462442700 {Asynchronous Received Call
Invoker}(CommandSubmitted) EvtSrc(Your_Halloween_Processor_Class_Name)
Command:move 100 -100
```

It is your choice whether you broadcast changes made in the server meta state to other clients based on a server user parameter set by calling broadcastMetastate() on the server. In case you decide to implement this feature, a test checks if you have implemented it.

Tags

The following are the expected tag combinations in the checks file:

The tags for the remote RMI interface provided by the server.

@DistributedTags.SERVER_REMOTE_INTERFACE+@DistributedTags.RMI,

The tags for the client out coupler class.

@DistributedTags.CLIENT_OUT_COUPLER+@DistributedTags.RMI+@DistributedTags.GIPC,

A single out coupler class can be used for capturing events of the local simulation. This class can delegate the sending of data via the right IPC mechanism to another class.
@DistributedTags.SERVER_REMOTE_INTERFACE+@DistributedTags.GIPC,

You can extend the server configurer class of the previous assignment to also configure GIPC objects.
@DistributedTags.SERVER_CONFIGURER+@DistributedTags.RMI+@DistributedTags.GIPC,

This is the same as in A4.
@DistributedTags.REGISTRY+@DistributedTags.RMI,

The RMI client remote interface – from A4:
@DistributedTags.CLIENT_REMOTE_INTERFACE+@DistributedTags.RMI,

The GIPC client remote interface is new:
@DistributedTags.CLIENT_REMOTE_INTERFACE+@DistributedTags.GIPC,

You can extend the client configurer class of the previous assignment to also configure GIPC objects.
@DistributedTags.CLIENT_CONFIGURER+@DistributedTags.RMI+@DistributedTags.GIPC,

A common server main class must support both RMI and GIPC:
@DistributedTags.SERVER+@DistributedTags.RMI+@DistributedTags.GIPC,

A common client main class must supports both RMI and GIPC:
@DistributedTags.CLIENT+@DistributedTags.RMI+@DistributedTags.GIPC,

A common client class can implement the remote methods called by RMI and GIPC client proxies:
@DistributedTags.CLIENT_REMOTE_OBJECT+@DistributedTags.RMI+@DistributedTags.GIPC,

A common server class can implement the remote methods called by RMI and GIPC server proxies:
@DistributedTags.SERVER_REMOTE_OBJECT+@DistributedTags.RMI+@DistributedTags.GIPC

As we see above, the RMI registry class has the same tags:
RMI registry main class:
@Tags({DistributedTags.REGISTRY, DistributedTags.RMI})

The server and client main classed have an additional tag to indicate they are also GIPC-bases server and client, respectively

Server main class:
@Tags({DistributedTags.SERVER, DistributedTags.RMI, DistributedTags.GIPC})

Client main class:
@Tags({DistributedTags.CLIENT, DistributedTags.RMI, DistributedTags.GIPC})

Ideally the server (client) interface should be the same interface, regardless of whether GIPC or RMI is being used for communication. The tag structure above gives you flexibility to create different interfaces for GIPC. In case you create, for example, the same server interface for RMI and GIPC, you put the following tags:

Server interface, unified interface:

@Tags({DistributedTags.SERVER_REMOTE_INTERFACE, DistributedTags.RMI, DistributedTags.GIPC})

The proxies generated from the interfaces will be different for RMI and GIPC.

Tracing

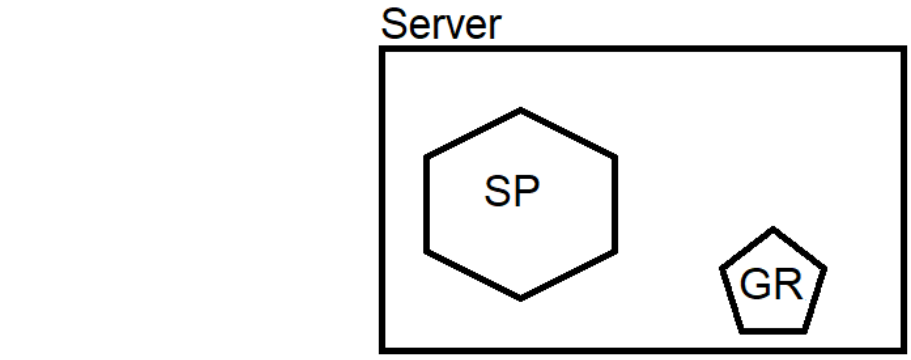
You must invoke the following code at the start of the server and each client:

```
FactoryTraceUtility.setTracing();
BeanTraceUtility.setTracing();
RMITraceUtility.setTracing();
ConsensusTraceUtility.setTracing();
ThreadDelayed.enablePrint();
GIPCRPCTraceUtility.setTracing();
NIOTraceUtility.setTracing();
```

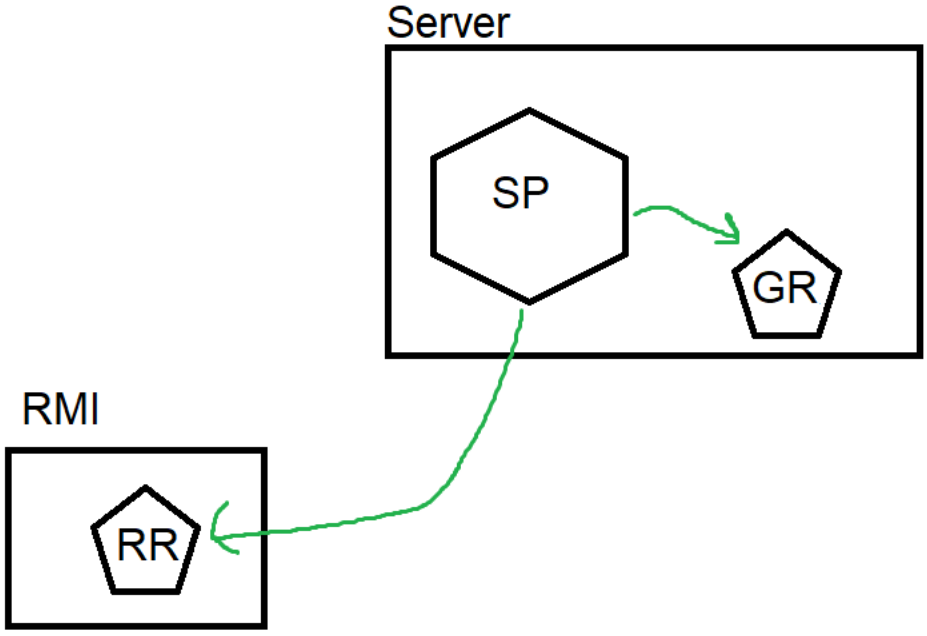
Image Visualization (Anna Wortas)

This describes what the extended server and main classes do to support both RMI and GIPC.

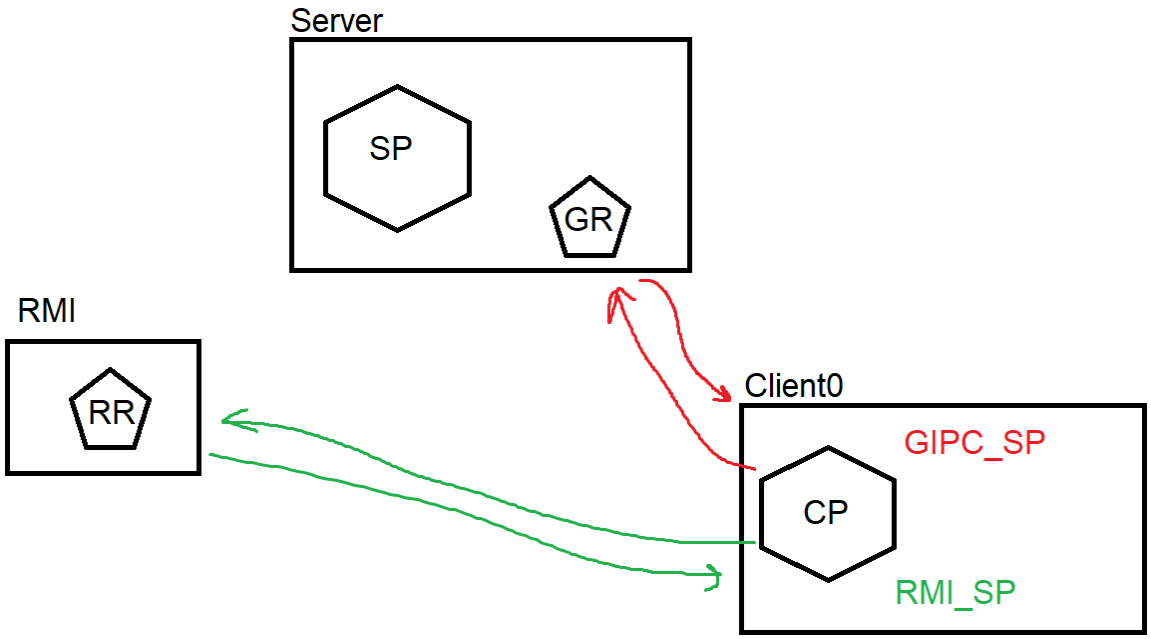
As in A4, the RMI registry main creates the RMI registry. As in A4, the server starts by creating its simulation proxy.



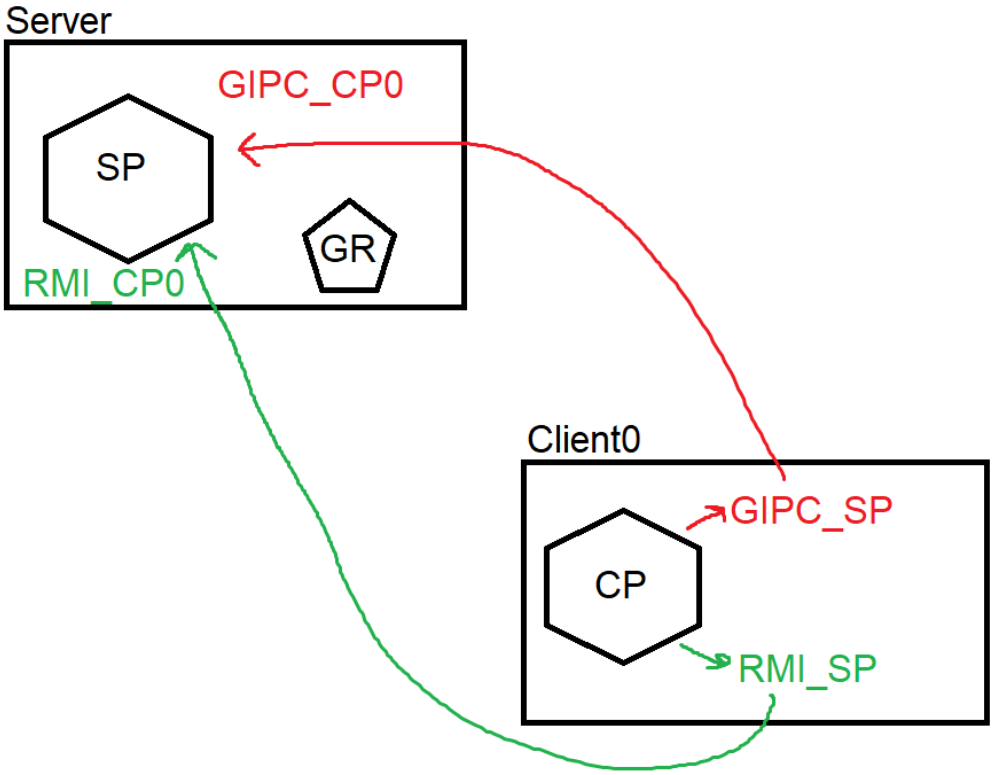
The server remote object registers itself with the gipc and rmi registries.



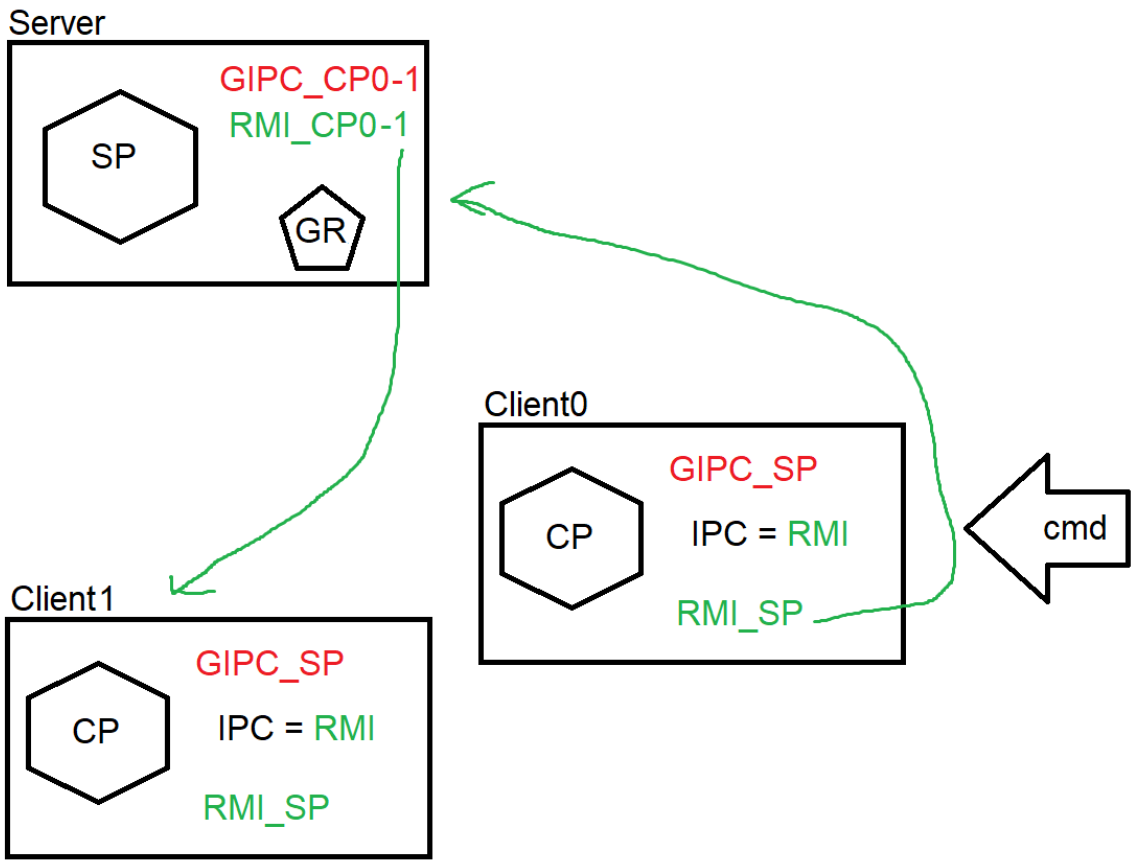
When the client initializes, it creates RMI and GIPC registry references, and retrieves/created both an RMI and GIPC server proxy



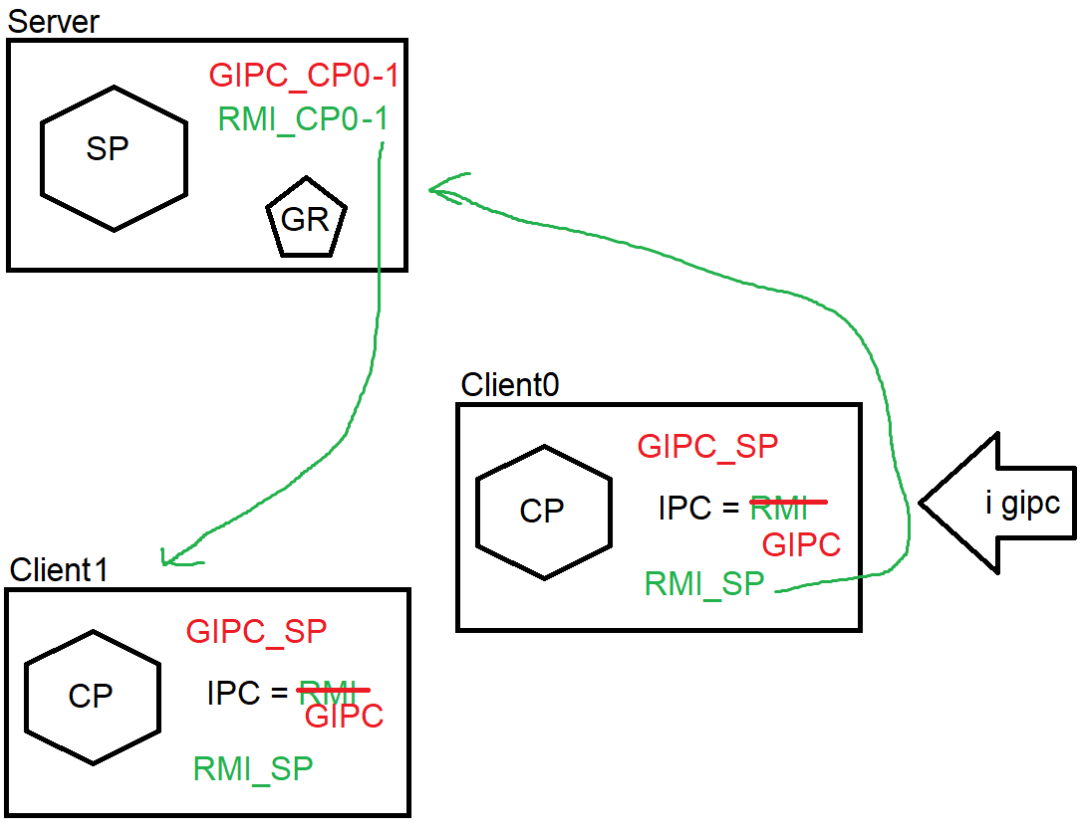
The client registers sends to the server both gipc and rmi proxies to receive data from the server The server stores the two proxies it receives. This registration is done using separate method calls in the server.



When a client receives a command, it sends that command to the server using the IPC mechanism currently selected. The server then uses its IPC mechanism to forward the message to all other clients.



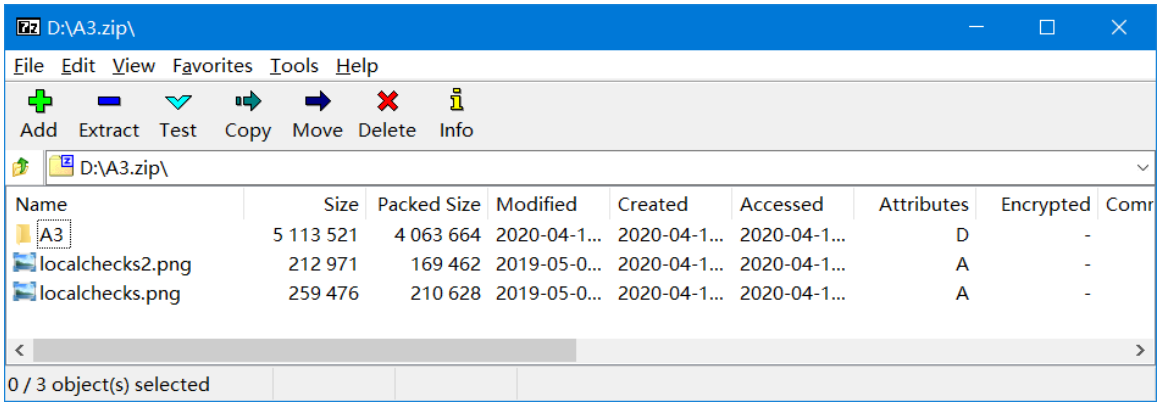
When “b true” and a “i gipc” are called in the console, the client that initially gets this message will update itself then forward the message to the server exclusively through RMI. This updates the IPC for all other clients connected.



Submission Instructions

Same as those for A4 .

As in A4, when zipping your submission, put your screenshots outside the project folder, select your project folder and your screenshots, and zip them. Do not put your screenshots inside your project folder, that is where all the images for the simulations are. Your zip file should look like this:



Key Issues and Components of Basic Implementation

GIPC: Server and Client Remote Objects

Like RMI, GIPC does not require client objects to be registered using rebind. Unlike RMI, it does not require registered objects to be exported, nor does it support a remote RMI registry. As both work with Java, they can create proxies for instances of the same class. The implementations of the proxies (created by the underlying system), however, are different.

As the proxies to client objects are different, you will need to create a separate object in the server to keep track of GIPC client proxies and call methods in them. The RMI and GIPC server relaying objects, however, can be of the same class – as they work with remote interfaces, they can tolerate different implementations of the interfaces provided by the two kinds of proxies.

Similarly, the GIPC and RMI client remote objects can be instances of the same class.

Thus, though your extended client and launchers will register with GIPC new client and server remote objects, no new code is needed to implement these GIPC remote objects.

GIPC: Server and Client Out Couplers

You cannot reuse for GIPC communication the RMI observing-sending object (out coupler) as it has a reference to an RMI proxy implementation. However, if you pass the IPC mechanism as a constructor argument, the GIPC and RMI client observing-sending objects can be of the same class. When a new command is observed, the object sends the message through its server proxy only if the current IPC mechanism is the one passed to its constructor.

GIPC: Client and Server Coupled Simulation Classes

Your client and server coupled simulation classes will need to perform all of the setup tasks they performed in the previous assignment. In addition, they will need to perform the setup tasks for GIPC - looking up a registry and registering and looking up registered objects. Again, we recommend subclassing - the class that does the setup for GIPC is simply a subclass of the one that does the setup for RMI, calling super to perform the RMI tasks. As in A4, *You must make sure that processCommands() is called once after all setup has been done.*

Synchronized Metastate

To maintain the meta state in the client and server, you can use the provided abstract bean class. Your subclass of the abstract class in the client program can override the methods that change the two components of the meta state. When this state is to be broadcast, these methods will call methods in the server relayer object instead of the superclass setter methods. This means your client callback object must define methods to invoke these setter methods in response to remote calls from the relayer object.

Race conditions can occur in a client between a thread that reads the broadcast mode and an RMI thread that writes to it in response to a remote RMI call. If you want to avoid

them, these two methods should be synchronized, which is the case in the provided abstract parameters bean class.

However, make sure that the method that sends the broadcast mode or IPC mechanism to the server is not a synchronized method– otherwise synchronous RMI calls can result in a deadlock (depending on which thread does the broadcast on the server).

Possible Issues

Cast Problems

You cannot cast a proxy to the class of the remote object, which may exist only at the client. It must be cast to an interface.

RPCOnUnregisteredObjectException

The problem occurs when your client is looking up the server object with a name before the server has registered the object with the name. They are using two different names if this is not a race condition. You may be passing the wrong args. Look at the log for traces about registration and lookup of objects in the server and client, respectively.

Other issues

Make sure the host is passed correctly to the client and server through the main args.

Use Args Processors rather than hard coded ports.

Use the right one - clients use ClientArgsProcessor, server uses ServerArgsProcessor., and registry uses RegistryArgsProcessor.

Make sure the right traces are turned on.

Make sure, as in A4, you have the if statement that checks to make sure the propertyName is InputString from the propertyChange() method of your observer.

Make sure that the send trace message is printed *before* and not *after* the send.

From Akshay Jain:

Easy to overlook mistake here guys, but your server instance needs to have two separate hashmaps where it registers rmi client objects and gipc client objects with separate registration methods for both. Otherwise, you'll end up with a mixed gipc/rmi process communication between two clients and server where the client with the initial command communicates with the server using gipc but then the server communicates with the other client using rmi because you were only storing the rmi client objects before.

Make sure that when the client tells the server to broadcast the IPC mechanism, make sure the server updates its own IPC mechanism in addition to broadcasting other clients.

Make sure you put NIOTraceUtility.setTracing(); in your code.

Traces needed for GIPCRMITwoCilentReadWriteAtomic

A lot of people seem to have trouble with this test so we are posting all the traces required for the GIPC part so you can check yourself. If you believe your console output has these traces, then include screenshots of the traces in your submission and we will handle it when grading. I separated the traces into parts so you may be able to deduce the traces needed for other tests.

When you see an error message, pay attention to the first few words as well as the thread name, same missing trace but different error message and different thread name indicate different error.

Server GIPC setup

```
{main}(GIPCRegistryCreated)

{main}(SelectorFactorySet)

{main}(ListenableAcceptsEnabled)

{main}(SelectorRequestEnqueued)

{main}(SelectorWokenUp)

{Selecting Thread}(SelectorRequestDequeued)

{Selecting Thread}(SocketChannelBlockingConfigured)

{Selecting Thread}(SocketChannelRegistered)

{Selecting Thread}(SelectCalled)

Client connect to server via GIPC stage 1

{main}(GIPCRegistryLocated)

{main}(SelectorFactorySet)

{main}(SelectorFactorySet)

{main}(SocketChannelConnectRequested)

{main}(SelectorRequestEnqueued)

{main}(SelectorWokenUp)

{Selecting Thread}(SelectorRequestDequeued)

{Selecting Thread}(SocketChannelBlockingConfigured)

{Selecting Thread}(SocketChannelConnectInitiated)

{Selecting Thread}(SocketChannelRegistered)

{Selecting Thread}(SelectCalled)

{Selecting Thread}(SocketChannelConnected)

{Selecting Thread}(SocketChannelInterestOp)
```

```
{Selecting Thread}(SocketChannelWriteRequested)
{Selecting Thread}(WriteRequestEnqueued)
{Selecting Thread}(WriteRequestEnqueued)
{Selecting Thread}(SelectorWokenUp)
{Selecting Thread}(SocketChannelInterestOp)
{Selecting Thread}(SelectCalled)
{Selecting Thread}(SocketChannelWritten)
{Selecting Thread}(WriteRequestDequeued)
{Selecting Thread}(SocketChannelWritten)
{Selecting Thread}(WriteRequestDequeued)
{Selecting Thread}(SocketChannelInterestOp)
{Selecting Thread}(WriteBufferIsEmpty)
{Selecting Thread}(ReadsEnabled)
{Selecting Thread}(SocketChannelInterestOp)
{Selecting Thread}(SelectCalled)
```

Client connect to server via GIPC stage 2

```
{main}(SocketChannelWriteRequested)
{main}(WriteRequestEnqueued)
{main}(WriteRequestEnqueued)
{main}(SelectorWokenUp)
{Selecting Thread}(SelectCalled)
{Selecting Thread}(SocketChannelWritten)
{Selecting Thread}(WriteRequestDequeued)
{Selecting Thread}(SocketChannelWritten)
{Selecting Thread}(WriteRequestDequeued)
```

Client send command to server via GIPC

```
{main}(NotifiedPropertyChangeEvent
{main}(RemoteProposeRequestSent
{main}(SocketChannelWriteRequested
```

{main}(WriteRequestEnqueued

{main}(WriteRequestEnqueued

{main}(SelectorWokenUp

{Selecting Thread}(SocketChannelInterestOp

{Selecting Thread}(SocketChannelWritten

{Selecting Thread}(WriteRequestDequeued

{Selecting Thread}(SocketChannelWritten

{Selecting Thread}(WriteRequestDequeued

{Selecting Thread}(SocketChannelInterestOp

{Selecting Thread}(WriteBufferIsEmpty

{Selecting Thread}(ReadsEnabled

{Selecting Thread}(SocketChannelInterestOp

Server receive command from client via GIPC

{Selecting Thread}(SelectUnblocked)

{Selecting Thread}(SocketChannelRead)

{Selecting Thread}(SocketChannelHeaderRead)

{Selecting Thread}(SocketChannelFullMessageRead)

{Asynchronous Received Call Invoker}(ReceivedCallInitiated)

{Asynchronous Received Call Invoker}(RemoteProposeRequestReceived)

Server send command to client via GIPC

{Asynchronous Received Call Invoker}(ProposalLearnedNotificationSent)

{Asynchronous Received Call Invoker}(SocketChannelWriteRequested)

{Asynchronous Received Call Invoker}(WriteRequestEnqueued)

{Asynchronous Received Call Invoker}(WriteRequestEnqueued)

{Asynchronous Received Call Invoker}(SelectorWokenUp)

{Selecting Thread}(SocketChannelInterestOp)

{Selecting Thread}(SocketChannelWritten)

{Selecting Thread}(WriteRequestDequeued)

{Selecting Thread}(SocketChannelWritten)

{Selecting Thread}(WriteRequestDequeued)

{Selecting Thread}(SocketChannelInterestOp)

{Selecting Thread}(WriteBufferIsEmpty)

Client receive command from server via GIPC

{Selecting Thread}(SelectUnblocked)

{Selecting Thread}(SocketChannelRead)

{Selecting Thread}(SocketChannelHeaderRead)

{Selecting Thread}(SocketChannelFullMessageRead)

{Selecting Thread}(SelectCalled)

{Asynchronous Received Call Invoker}(ReceivedCallInitiated)

{Asynchronous Received Call Invoker}(ProposalLearnedNotificationReceived)

GPT Keypoints and Checklist

Can you please summarize the uploaded assignment indicating its key points?

ChatGPT said:

Here's a summary of the key points from **Assignment 5: Basic and Meta Object Replication Using GIPC RPC Abstractions** (Comp 533):

🧠 **Objective**

- Understand **synchronous vs. asynchronous RPCs.**
- Implement and replicate **meta state** (IPC mechanism selection) using both **RMI** and **GIPC.**
- Extend previous assignments by integrating GIPC support alongside RMI.

🔁 **Replication Scope**

- **Basic Objects Replicated (Already Done):**
    - Simulation object

    - Command processor object
- **Meta Object to be Replicated Now:**
    - IPC mechanism choice (RMI or GIPC)
    - Only replicated using **RMI**

🔌 **Key Differences: RMI vs. GIPC**

- **RMI** uses sockets; **GIPC** uses **NIO (non-blocking I/O)**.
- **RMI** methods are synchronous; **GIPC** methods that return void are asynchronous.
- **GIPC** requires:
    - Clients to have globally unique names
    - Explicit port settings via ServerArgsProcessor and ClientArgsProcessor
- Proxies in **GIPC** are always created on the client side.

🧱 **Implementation Highlights**

- Add IPC switching functionality via console commands:
    - i GIPC or Loci RMI – changes the active communication mechanism.
- Handle broadcastMetaState (console command b true) to:
    - Replicate IPC mechanism changes across clients and server using **RMI.**
    - Clients and server each hold a synchronized local copy.

🔍 **Methods to Override**

- void ipcMechanism(IPCMechanism newValue)
- void broadcastMetaState(boolean newValue)

These will manage local state and propagate changes depending on the broadcastMetaState flag.

🖥️ **Tagging Structure**

Include the correct @DistributedTags in all relevant classes/interfaces (e.g., SERVER_REMOTE_INTERFACE, CLIENT_OUT_COUPLER, GIPC, RMI, etc.) to pass automated checks.

🧪 Testing and Tracing

Enable all required tracing utilities:

java

CopyEdit

```
FactoryTraceUtility.setTracing();

BeanTraceUtility.setTracing();

RMITraceUtility.setTracing();

ConsensusTraceUtility.setTracing();

ThreadDelayed.enablePrint();

GIPCRPCTraceUtility.setTracing();

NIOTraceUtility.setTracing();
```

Watch out for key trace markers during events such as:

- GIPC connection setup
- Sending/receiving commands
- Broadcasting IPC mechanism changes

🔄 Architecture Adjustments

- Maintain **separate registries/data structures** for RMI and GIPC proxies.
- Clients and server use **common simulation classes** that handle both protocols.
- Extend previous configurer and main classes for GIPC functionality.

⚠️ Potential Issues

- Avoid ClassCastException by always casting proxies to **interfaces**, not classes.
- Ensure the server updates its own IPC mechanism when broadcasting.
- Don’t hardcode ports—use provided argument processors.

- Watch out for synchronization issues—some methods should *not* be synchronized to prevent RMI deadlocks.

Would you like a checklist to help guide your implementation or highlight the expected deliverables and traces?

4o

You said:

Yes

ChatGPT said:

Great! Here's a detailed **checklist** to guide your implementation of Assignment 5. I've broken it down into key components:

✅ Assignment 5 Implementation Checklist

🔌 1. GIPC Integration

- Add GIPC support to both client and server.
- Create and use GIPCRegistry with port from ServerArgsProcessor.getGIPCServerPort() on the server.
- Use GIPCLocateRegistry.getRegistry(host, port, clientName) on the client.
- Ensure clients have **globally unique names**.

🔁 2. Meta State: IPC Mechanism

- Create a local ipc_mechanism variable using provided abstract bean (AnAbstractSimulationParametersBean).
- Override void ipcMechanism(IPCMechanism newValue) in both client and server.
- Use ProposalMade and ProposalSet trace classes to log changes.
- Switch communication mode (RMI or GIPC) based on this variable when sending simulation commands.

📡 3. Dynamic IPC Switching

- Support console command i GIPC and i RMI to change IPC mechanism dynamically.

- Ensure only **sending clients** (out couplers) use the selected IPC mode.

📣 4. Broadcasting Meta State

- Override void broadcastMetaState(boolean newValue) in client class.
- Respond to console command b true to enable future broadcasts.
- When enabled, propagate IPC mechanism changes via **RMI** to:
    - Other clients
    - Server
- Server must also update its own ipc_mechanism.

🧩 5. Class Tags

Ensure correct @DistributedTags are used:

Component	Required Tags
Server remote interface	SERVER_REMOTE_INTERFACE, RMI, GIPC
Client remote interface	CLIENT_REMOTE_INTERFACE, RMI, GIPC
Server main class	SERVER, RMI, GIPC
Client main class	CLIENT, RMI, GIPC
Registry	REGISTRY, RMI
Client out coupler	CLIENT_OUT_COUPLER, RMI, GIPC
Server configurer	SERVER_CONFIGURER, RMI, GIPC
Client configurer	CLIENT_CONFIGURER, RMI, GIPC
Server remote object	SERVER_REMOTE_OBJECT, RMI, GIPC
Client remote object	CLIENT_REMOTE_OBJECT, RMI, GIPC

🔍 6. Required Tracing Setup

At startup (in server and all clients), include:

java

CopyEdit

```
FactoryTraceUtility.setTracing();

BeanTraceUtility.setTracing();

RMITraceUtility.setTracing();

ConsensusTraceUtility.setTracing();

ThreadDelayed.enablePrint();

GIPCRPCTraceUtility.setTracing();

NIOTraceUtility.setTracing();
```

📸 7. Screenshot & Trace Checklist

You must capture traces for the following:

GIPC Setup Traces

- Server: GIPCRegistryCreated, SelectorFactorySet, etc.
- Client: GIPCRegistryLocated, SocketChannelConnectRequested, etc.

IPC Switching

- ProposalMade and ProposedStateSet when IPC is changed
- RemoteProposeRequestSent and RemoteProposeRequestReceived (when broadcasting)

Command Sending

- CommandSubmitted, RemoteProposeRequestSent, SocketChannelWriteRequested, etc.

Receiving Command

- Server: RemoteProposeRequestReceived
- Client: ProposalLearnedNotificationReceived, CommandSubmitted

You can refer to the end of the assignment document for the full expected trace log if needed.

⚠️ 8. Implementation Pitfalls to Avoid

- Don’t mix GIPC and RMI client objects in the same data structure.
- Avoid using synchronized on methods making RMI calls to prevent deadlocks.
- Always cast proxies to **interfaces**, not concrete classes.
- Ensure processCommands() is called *after* all setup is complete.

📦 9. Submission Structure

- Project folder (code and resources)
- Screenshots (separately, not inside the project folder)
- Zip both together

Would you like this in a downloadable format (like PDF or checklist template), or help automating the trace validation?